

Phân tích bài toán

1. Bài toán

Quy trình thuê nhà truyền thống (giấy tờ, tiền mặt/chuyển khoản ngân hàng) có ba điểm yếu chính:

- Tiền cọc thiếu minh bạch** — chủ nhà giữ tiền cọc, người thuê phải tin tưởng chủ nhà sẽ hoàn cọc đúng hạn/đúng số tiền khi kết thúc hợp đồng.
- Không có bằng chứng khách quan** — việc "đã trả tiền thuê kỳ này chưa", "đã bàn giao phòng chưa" thường chỉ dựa vào lời nói hoặc tin nhắn, khó tra cứu lại khi có tranh chấp.
- Xử lý thủ công, tốn thời gian** — chủ nhà phải tự theo dõi từng người thuê, từng kỳ thanh toán, tự tính toán khấu trừ cọc.

2. Giải pháp

Đưa 3 nghiệp vụ lõi lên smart contract để một bên trung gian **không thể thiên vị** (chính là đoạn mã đã triển khai, không ai sửa được) đảm nhiệm:

- Giữ tiền cọc khách quan:** tiền cọc do smart contract giữ (không vào ví chủ nhà ngay), chỉ giải ngân theo đúng luật đã lập trình sẵn.
- Ghi nhận giao dịch không thể chỉnh sửa:** mọi lần đặt cọc, trả tiền, bàn giao, tất toán đều là các *event* trên blockchain — tra cứu lại được vĩnh viễn, không ai xoá/sửa.
- Tự động hoá luật hợp đồng:** các điều kiện (chỉ người thuê mới được trả tiền, không được khấu trừ vượt quá cọc, chủ nhà không thể tự thuê nhà mình...) được smart contract tự kiểm tra và từ chối giao dịch sai ngay lập tức, không cần trọng tài con người.
- Bằng chứng sở hữu hợp đồng bằng token:** mỗi lượt thuê thành công được đại diện bằng một token ERC-721 không chuyển nhượng (`RentalAgreementToken`), mint thẳng cho người thuê ngay khi đặt cọc — tách biệt "ai đang giữ quyền của hợp đồng thuê nào" (token) ra khỏi "luật nghiệp vụ thuê nhà" (`RentalManager`), đúng mô hình token/manager theo quy trình chuẩn. Lý do chọn ERC-721 (thay vì ERC-20/1155) và thiết kế `tokenId == propertyId` : xem [lua-chon-token.md](#).
- Tra cứu nhanh không phụ thuộc trực tiếp vào node:** một backend indexer nghe event từ contract, ghi lại vào SQLite, expose REST API — blockchain vẫn là nguồn dữ liệu có thẩm quyền, backend chỉ là lớp chỉ mục giúp tra cứu nhanh hơn gọi lại `eth_call` mỗi lần. Chi tiết: [backend/README.md](#).

3. Đối tượng người dùng (actor)

Actor	Vai trò
Chủ nhà (Landlord)	Đăng tài sản cho thuê, nhận tiền thuê định kỳ, quyết định khấu trừ cọc khi kết thúc hợp đồng.
Người thuê (Tenant)	Đặt cọc để kích hoạt hợp đồng, trả tiền thuê định kỳ, xác nhận bàn giao khi trả phòng, có thể khiếu nại nếu không đồng ý mức khấu trừ.

Actor	Vai trò
Trọng tài (Arbiter)	Địa chỉ được cấp <code>ARBITER_ROLE</code> (qua <code>grantRole / scripts/grant-arbiter.ts</code>) — bỏ phiếu quyết định mức khấu trừ khi có tranh chấp, chỉ có hiệu lực khi đủ số trọng tài độc lập đồng thuận (multisig). Không tham gia nghiệp vụ đăng tin/thuê/trả tiền.
(Người xem công khai)	Bất kỳ ai cũng đọc được lịch sử giao dịch trên blockchain (tính minh bạch), nhưng không có quyền ghi.

4. Phạm vi chức năng đã triển khai

4.1. Nghiệp vụ thuê nhà (mức tối thiểu)

- ✓ Chủ nhà đăng thông tin tài sản (`listProperty`)
- ✓ Người thuê đặt cọc (`rentProperty`) — đồng thời **tạo hợp đồng thuê** (kích hoạt trạng thái `Active` , không có bước "tạo hợp đồng" tách riêng vì đặt cọc chính là hành động xác nhận hợp đồng bắt đầu) và **mint token đại diện hợp đồng** cho người thuê trong cùng giao dịch
- ✓ Thanh toán tiền thuê định kỳ (`payRent`)
- ✓ Xác nhận bàn giao (`confirmHandover`)
- ✓ Hoàn hoặc khấu trừ tiền đặt cọc + kết thúc hợp đồng (`proposeSettlement` → `acceptSettlement` , hoặc → `disputeSettlement` → `voteOnDispute` nếu tranh chấp — xem mục 4.5)
- ✓ Theo dõi lịch sử thanh toán (đọc trực tiếp từ event trên blockchain, tab "Lịch sử")
- ✓ (Bổ sung ngoài yêu cầu) Huỷ tin đăng nhằm (`cancelListing`) — chủ nhà huỷ được tin còn trống (chưa ai đặt cọc) khi lỡ đăng sai (vd sai giá); giải quyết hạn chế thực tế là các trường của `Property` không sửa lại được sau khi ghi on-chain.

4.2. Token & tách kiến trúc (Bước 0–2 của quy trình)

- ✓ Tách riêng **token contract** (`RentalAgreementToken` , ERC-721 không chuyển nhượng, `AccessControl` với `MINTER_ROLE`) khỏi **business contract** (`RentalManager` , không dùng `AccessControl` — thị trường không cần cấp phép)
- ✓ Ignition module deploy đúng thứ tự: token → manager → cấp `MINTER_ROLE` cho manager

4.3. Toolchain (bám theo tài liệu mẫu)

- ✓ Hardhat 3 + TypeScript (ESM), Hardhat Ignition thay cho script deploy thủ công, Hardhat keystore thay cho `.env` khi deploy Sepolia

4.4. Backend & bảo mật (Bước 4)

- ✓ Backend event indexer + REST API (SQLite qua `node:sqlite` , xem [backend/README.md](#))
- ✓ Rà soát bảo mật (Slither + rà soát thủ công thay thế khi Slither không chạy được trên Hardhat 3, xem [bao-mat-slither.md](#)), test bảo mật (chặn `safeTransferFrom` , chặn reentrancy qua `onERC721Received`)
- ✓ Checklist production ghi rõ hạng mục còn thiếu, không tick giả — xem [production-checklist.md](#)

4.5. Chức năng nâng cao theo đề bài

- ☒ **Cơ chế trọng tài khi có tranh chấp:** `proposeSettlement` → `acceptSettlement` (đồng ý) hoặc `disputeSettlement` (khiếu nại) → `voteOnDispute` (trọng tài xử lý). Chi tiết luồng và lý do thiết kế: [gioi-han-va-rui-ro.md § 3.1](#).
- ☒ **Phạt thanh toán trễ:** `payRent` bắt buộc cộng thêm phần trăm phạt (`lateFeeBps` , mặc định 5%) nếu gọi sau `nextDueDate` . Chi tiết: [gioi-han-va-rui-ro.md § 3.2](#).
- ☒ **Multisig cho giải ngân tiền cọc:** khi tranh chấp, cần đủ số trọng tài (`arbiterApprovalsRequired` , mặc định 2) **độc lập đồng thuận cùng 1 mức khấu trừ** mới thi hành — không dùng `TimeLockController` (đề xuất ban đầu) mà đếm phiếu trực tiếp on-chain, đơn giản hơn và không cần contract phụ trợ. Đánh đổi: không có độ trễ thời gian giữa lúc đủ phiếu và lúc thực thi — xem [gioi-han-va-rui-ro.md § 4](#).

Quản trị vai trò trọng tài: `admin` (ví deploy) mặc định cũng là trọng tài #1 — trong quá trình test phát sinh vấn đề xung đột lợi ích thật (admin đồng thời là chủ nhà, tự bỏ phiếu được cho tranh chấp của chính mình). Đã khắc phục bằng quản trị vai trò (`scripts/revoke-arbiter.ts` thu hồi quyền khỏi admin, `scripts/grant-arbiter.ts` cấp cho ví độc lập) — chi tiết và giới hạn còn lại (chưa chặn ở mức contract) xem [gioi-han-va-rui-ro.md § 4](#).

Các hạng mục còn thiếu cho production thật (Foundry fuzz test, audit độc lập, `Pausable` , multisig admin, PostgreSQL) được liệt kê đầy đủ tại [production-checklist.md](#).